



Taller Práctico: Árboles AVL y Detección de Desbalances

Evaluación de Implementación de Rotaciones y Propiedades de Altura en C

1. OBJETIVOS & PREÁMBULO DE ENTIDADES

El objetivo de este taller es consolidar el manejo de estructuras auto-balanceadas en altura (AVL), asegurando la correcta manipulación de punteros ascendentes (*parent*) y descendentes (*left*, *right*), así como el diseño de algoritmos recursivos *bottom-up*.

Para este ejercicio, cada nodo del árbol mantiene su factor implícito o calculado mediante alturas y conexiones explícitas a su predecesor:

```
typedef struct avl_node { int key; // Clave o valor almacenado int height; // Altura del nodo
```

2. SECCIÓN GUIADA: ROTATE-RIGHT (TIEMPO EST.: 20 MIN)

Considere un nodo y cuyo hijo izquierdo es x . La rotación simple a la derecha reorganiza el subárbol manteniendo la invariante de búsqueda simétrica ($T_1 < x < T_2 < y < T_3$) y elevando el nodo x a la posición que ocupaba y .



Debes implementar en C la función con la siguiente firma:

```
AvlNode* avl_rotate_right(AvlTree *tree, AvlNode *y);
```

Pseudocódigo oficial de la operación:

```
avl_rotate_right(tree, y): 1:  $x \leftarrow y.\text{left}$  2:  $T2 \leftarrow x.\text{right}$  3:  $y.\text{left} \leftarrow T2$  4: if  $T2 \neq \text{NULL}$  then
```

Regla de consistencia: Recuerde que si y era la raíz del árbol general ($\text{tree} \rightarrow \text{root}$), la estructura contenedora debe quedar apuntando a x como nueva raíz.

3. SECCIÓN AUTÓNOMA: FIND-UNBALANCED (TIEMPO EST.: 40 MIN)

En un árbol binario balanceado en altura según el criterio AVL, se define para cada nodo u el factor de balance $BF(u) = h(u.left) - h(u.right)$. La propiedad fundamental exige que:

$$-1 \leq BF(u) \leq 1 \quad \forall u \in Tree$$

Cuando se insertan o eliminan llaves sin rebalancear, pueden generarse nodos con $|BF(u)| \geq 2$. Su tarea es diseñar e implementar un algoritmo recursivo que identifique el **nodo desbalanceado más profundo** (es decir, el primer nodo con violación que se detecta subiendo desde las hojas).

```
AvlNode* avl_find_unbalanced(AvlNode *node, int *height);
```

Especificación y Requerimientos de la Función

- **Parámetro de altura por referencia ($int *height$):** Similar a los patrones acumuladores usados con índices en talleres previos, la función debe reportar la altura calculada del subárbol actual hacia la llamada superior. Se asume que un nodo nulo ($NULL$) tiene altura -1 y un nodo hoja tiene altura 0 .
- **Post-Order / Bottom-Up:** Antes de diagnosticar el nodo actual, se deben evaluar recursivamente sus subárboles izquierdo y derecho.
- **Criterio de propagación:** Si el subárbol izquierdo o derecho ya contiene un nodo desbalanceado, dicho puntero debe propagarse directamente hacia arriba sin sobrescribirlo con ancestros desbalanceados.
- **Retorno:** Si se encuentra un desbalance en el nodo actual ($|h_{izq} - h_{der}| > 1$), se retorna dicho nodo. Si todo el subárbol está balanceado, se retorna $NULL$.

4. FORMATO DE ARCHIVO DE ENTRADA Y SALIDA

El programa recibe el número de eventos E , seguido de comandos secuenciales con arreglos de llaves enteros:

Archivo de Entrada (input.txt)	Archivo de Salida Esperado (output.txt)
3 ROTATE 5 7 4 9 2 5 CHECK-UNBALANCED 4 10 20 30 40 CHECK-UNBALANCED 3 20 10 30	Rotacion derecha realizada sobre nodo 4. Nueva sub-raiz: 2 Primer desbalance detectado en nodo: 20 Arbol balanceado correctamente

Advertencia de Compilación & Valgrind: Asegúrese de inicializar las alturas correctamente antes de la desreferenciación de punteros. Cualquier acceso a memoria no inicializada sobre *parent* provocará un *Segmentation Fault* inmediato durante las pruebas automatizadas.

5. PAUTA DE AUTOEVALUACIÓN EN 60 MINUTOS

[00 - 20 min]	Traducir e implementar <i>avl_rotate_right</i> . Verificar casos borde ($y \rightarrow parent == NULL$ y $T2 == NULL$).
[20 - 50 min]	Diseñar e implementar el recorrido bottom-up en <i>avl_find_unbalanced</i> calculando alturas con $*height = 1 + \max(h1, h2)$.
[50 - 60 min]	Compilar con <i>make</i> , probar contra casos de prueba y contrastar con <i>diff</i> .